



72.41 - Base de Datos II

Trabajo Práctico Obligatorio Sistema de Gestión de Clínica Veterinaria

Primer cuatrimestre de 2026

Alumnos:

Ian Erlij ()

Federico Saroka (64213)

Gerónimo Fretes (64988)

Repositorio: <https://github.com/GeronimoFretes/vetsalud-nosql>

Fecha de entrega: 12/06/2026

Sistema de Gestión de Clínica Veterinaria.....	0
1. Introducción.....	2
2. Arquitectura elegida.....	2
3. Justificación técnica de los motores.....	2
3.1. MongoDB.....	2
3.2. Redis.....	3
4. Modelo de datos.....	3
4.1. Colección propietarios.....	3
4.2. Colección pacientes.....	4
4.3. Colección veterinarios.....	4
4.4. Colección consultas.....	4
4.5. Colección vacunaciones.....	5
4.6. Colección cirugías.....	5
4.7. Stock farmacéutico en Redis.....	5
5. Carga de datos.....	5
6. Servicios implementados.....	6
7. Resultados de ejemplo.....	9
8. Ejecución y reproducibilidad.....	10
9. Conclusión.....	10
10. Bibliografía.....	10

1. Introducción

El objetivo del trabajo fue diseñar e implementar una solución de persistencia NoSQL para VetSalud S.A., una clínica veterinaria que busca reemplazar el uso de planillas por una arquitectura de datos más organizada, reproducible y escalable.

La solución implementada utiliza persistencia políglota, combinando dos motores NoSQL de paradigmas diferentes. MongoDB se utiliza como base documental principal para la información clínica y administrativa, mientras que Redis se utiliza para el manejo del stock farmacéutico.

El proyecto incluye la carga de datos desde los archivos CSV provistos, la incorporación de registros adicionales, la implementación de los 15 servicios solicitados y la generación de salidas JSON para documentar los resultados.

2. Arquitectura elegida

La arquitectura utiliza dos motores NoSQL.

MongoDB almacena las siguientes colecciones.

- **propietarios**
- **pacientes**
- **veterinarios**
- **consultas**
- **vacunaciones**
- **cirugias**

Redis almacena el stock farmacéutico mediante hashes, sorted sets y una lista de movimientos.

- **stock:producto:<id_producto>**
- **idx:stock:unidades**
- **movimientos:stock**

La decisión busca asignar cada parte del dominio al motor que mejor responde a sus patrones de acceso. MongoDB se adapta mejor a los datos clínicos y administrativos, donde se necesitan consultas por múltiples campos, agregaciones, filtros por fecha y relaciones por referencia. Redis se adapta mejor al stock, donde se necesitan accesos rápidos por clave, actualización de cantidades y consulta eficiente de productos con bajo stock.

3. Justificación técnica de los motores

3.1. MongoDB

MongoDB fue elegido como motor principal porque el dominio clínico de VetSalud contiene entidades con estructura flexible y relaciones frecuentes entre documentos. Las consultas requeridas combinan pacientes, propietarios, veterinarios, consultas y vacunaciones. Para resolverlas se utilizan operaciones de filtrado, agregación y `$lookup`.

Se eligió un modelo principalmente referenciado. Por ejemplo, cada paciente guarda el `id_propietario`, cada consulta guarda el `id_paciente` y el `id_vet`, y cada vacunación guarda el paciente y el veterinario asociados.

Esta decisión evita duplicar información sensible o mutable. Por ejemplo, si los datos de un propietario cambian, se actualiza un único documento en la colección `propietarios`. Esto es especialmente importante porque el trabajo exige un ABM completo de propietarios con modificación de datos y baja lógica.

Se descartó un modelo fuertemente embebido porque habría generado duplicación de propietarios, veterinarios o historias clínicas dentro de otros documentos. Aunque el embedding puede ser útil cuando los datos se consultan siempre juntos, en este caso las entidades también se consultan de forma independiente.

3.2. Redis

Redis fue elegido para el stock farmacéutico. Cada producto se guarda como un hash y sus unidades disponibles se indexan en un sorted set. Esto permite resolver directamente la consulta de productos con menos de 50 unidades.

Cada producto se modela así.

```
stock:producto:PRX001
  id_producto
  nombre
  categoria
  unidades
  precio_unit
  vencimiento
  proveedor
```

El índice de unidades se modela así.

```
idx:stock:unidades
  member = id_producto
  score = unidades
```

Esta estructura permite consultar productos con bajo stock mediante `ZRANGEBYSCORE` y luego recuperar los datos completos con `HGETALL`.

También se implementó una actualización masiva de stock después de una consulta. La actualización decrementa las unidades del hash, actualiza el sorted set y registra el movimiento en una lista de Redis. Esto muestra que Redis no se usa como un simple cache, sino como motor persistente para un caso funcional específico.

4. Modelo de datos

4.1. Colección propietarios

Cada propietario contiene datos de identificación y contacto. Además se incorpora el campo `activo`, necesario para implementar la baja lógica.

Campos principales.

```
id_propietario
nombre
apellido
```

dni
email
telefono
ciudad
provincia
activo

4.2. Colección pacientes

Cada paciente representa un animal registrado en la clínica.

Campos principales.

id_paciente
nombre
especie
raza
fecha_nac
id_propietario
activo

La relación con propietarios se resuelve mediante `id_propietario`.

4.3. Colección veterinarios

Cada veterinario contiene datos profesionales, especialidad, sucursal y estado activo.

Campos principales.

id_vet
nombre
apellido
matricula
especialidad
sucursal
activo

4.4. Colección consultas

Cada consulta médica referencia a un paciente y a un veterinario.

Campos principales.

id_consulta
id_paciente
id_vet
fecha
motivo
tipo_consulta
diagnostico
costo

estado

El campo `tipo_consulta` se deriva durante la carga de datos a partir del campo `motivo`. Esto permite resolver de forma más clara la consulta de controles con costo menor a 5000.

4.5. Colección vacunaciones

Cada vacunación referencia a un paciente y a un veterinario.

Campos principales.

```
id_vacuna
id_paciente
id_vet
fecha_aplicacion
nombre_vacuna
proxima_dosis
```

El campo `proxima_dosis` permite detectar vacunas vencidas.

4.6. Colección cirugías

Se agregó una colección de cirugías para cubrir el dominio funcional mencionado en el enunciado. Aunque no hay un servicio específico asociado a cirugías, la colección completa el modelo clínico.

Campos principales.

```
id_cirugia
id_paciente
id_vet
fecha
tipo
resultado
costo
```

4.7. Stock farmacéutico en Redis

El stock se separa del modelo clínico y se almacena en Redis. Se utilizan tres estructuras.

```
stock:producto:<id_producto>
idx:stock:unidades
movimientos:stock
```

La primera guarda los datos completos del producto, la segunda permite buscar por cantidad disponible y la tercera registra movimientos de stock.

5. Carga de datos

La carga de datos se realiza mediante scripts reproducibles. Los archivos CSV originales se ubican en `data/raw/` y se importan con los scripts del módulo `src.seed`.

El comando principal de carga es el siguiente.

```
python -m src.seed.seed_all
```

Durante la carga se normalizan tipos de datos. Las fechas se convierten a objetos de fecha, los costos y precios a valores numéricos, y los campos booleanos a valores lógicos. También se agregan registros adicionales para cumplir con el requisito de incorporar al menos 10 registros propios por colección o tabla.

Los registros adicionales fueron diseñados para asegurar que los servicios devuelvan resultados significativos. Se incorporaron propietarios con más de un paciente, consultas recientes, consultas en seguimiento, controles con costo menor a 5000, vacunas vencidas, productos con bajo stock y movimientos posibles de stock.

6. Servicios implementados

Nº	Servicio	Motor	Operación principal	Salida
1	Pacientes activos con datos de propietario	MongoDB	Filtra pacientes activos y agrega datos del propietario con \$lookup	service_01_pacientes_activos.json
2	Consultas en estado Seguimiento con veterinario asignado y costo	MongoDB	Filtra consultas por estado y agrega paciente y veterinario	service_02_consultas_seguimiento.json
3	Historial completo de un paciente	MongoDB	Combina consultas y vacunaciones de un paciente y ordena los eventos por fecha	service_03_historial_paciente.json

4	Propietarios con más de un paciente registrado	MongoDB	Agrupar pacientes por propietario, cuenta registros y agrega datos del propietario	service_04_propietarios_multiples_pacientes.json
5	Veterinarios activos y cantidad de consultas en los últimos 60 días	MongoDB	Filtrar veterinarios activos y cuenta consultas recientes mediante \$lookup con pipeline	service_05_veterinarios_consultas_60_dias.json
6	Pacientes con vacunas vencidas	MongoDB	Filtrar vacunaciones con próxima dosis anterior a la fecha actual y agrega paciente y veterinario	service_06_vacunas_vencidas.json
7	Top 5 diagnósticos más frecuentes	MongoDB	Agrupar consultas por diagnóstico, cuenta ocurrencias y ordena de mayor a menor	service_07_top_diagnosticos.json
8	Productos con menos de 50 unidades y proveedor	Redis	Busca productos en un sorted set por cantidad disponible y recupera detalles desde hashes	service_08_stock_bajo.json

9	Consultas de tipo Control con costo menor a 5000	MongoDB	Filtra consultas por tipo_consulta = Control y costo < 5000	service_09_controles_bajo_costo.json
10	Pacientes de una sucursal determinada a través del veterinario	MongoDB	Recorre consultas, veterinarios y pacientes para identificar pacientes atendidos en una sucursal	service_10_pacientes_por_sucursal.json
11	Ingresos totales por veterinario en el mes actual	MongoDB	Filtra consultas del mes actual, agrupa por veterinario y suma costos	service_11_ingresos_mes_actual.json
12	Propietarios sin consultas registradas en el último año	MongoDB	Busca propietarios cuyos pacientes no tienen consultas recientes	service_12_propietarios_sin_consultas_anio.json
13	ABM completo de propietarios	MongoDB	Implementa alta, modificación de datos y baja lógica mediante activo = false	service_13_abm_propietarios.json
14	Registro de nueva consulta médica con validación	MongoDB	Valida paciente y veterinario existentes y activos antes de insertar la consulta	service_14_registrar_consulta.json

15	Actualización masiva de stock tras una consulta	Redis	Decrementa unidades en hashes, actualiza el sorted set y registra movimientos de stock	service_15_actualizar_stock.json
----	---	-------	--	----------------------------------

7. Resultados de ejemplo

Nº	Resultado de ejemplo
1	Paciente PEX001: Mora, especie Perro, raza Labrador, propietario Lucia Benitez.
2	Consulta CEX002: estado Seguimiento, paciente Tango, veterinario Bruno Salvatierra, costo \$7.600.
3	Paciente PEX001: historial con consulta de control general, vacunación antirrábica y consulta de control neurológico ordenadas por fecha.
4	Propietario PRA001: Lucia Benitez, con 2 pacientes registrados, Mora y Tango.
5	Veterinario VEX001: Ana Pereyra, activo, con consultas registradas dentro de los últimos 60 días.
6	Vacuna Antirrábica vencida para el paciente PEX001, con próxima dosis anterior a la fecha actual.
7	Diagnóstico frecuente: Sano, identificado entre los diagnósticos con mayor cantidad de consultas.
8	Producto PRX006: Anestésico, 15 unidades disponibles, proveedor VetSur.
9	Consulta CEX005: tipo Control, costo \$3.900, menor al límite de \$5.000.
10	Sucursal Palermo: paciente PEX001 identificado a través del veterinario que lo atendió en esa sucursal.
11	Veterinario VEX004: ingresos del mes actual calculados a partir de las consultas registradas en el período.
12	Propietario PRA006: Federico Lopez, con paciente registrado y sin consultas en el último año.
13	Propietario PABM001: creado, modificado y luego dado de baja lógicamente con <code>activo = false</code> .

- 14 Consulta CNUEVA001: registrada para el paciente Mora luego de validar paciente y veterinario existentes y activos.
- 15 Consulta CNUEVA001: producto PRX003 decrementado de 120 a 118 unidades en Redis.

8. Ejecución y reproducibilidad

El proyecto puede ejecutarse desde cero con los siguientes comandos.

```
docker compose up -d
python -m pip install -r requirements.txt
cp .env.example .env
python -m src.run_all_services --reset
python -m src.validate_project
```

El comando `run_all_services --reset` reconstruye las bases desde los CSV y los datos adicionales, ejecuta los 15 servicios y genera las salidas JSON. El comando `validate_project` verifica que las colecciones tengan datos suficientes, que Redis tenga stock cargado, que existan las salidas de los servicios y que los 15 servicios hayan finalizado correctamente.

La validación genera el archivo.

```
outputs/validation_summary.json
```

9. Conclusión

La solución implementa una arquitectura NoSQL políglota para VetSalud, utilizando MongoDB y Redis con responsabilidades diferenciadas.

MongoDB concentra el dominio clínico y administrativo, resolviendo consultas sobre pacientes, propietarios, veterinarios, consultas y vacunaciones. Redis concentra el dominio de stock farmacéutico, resolviendo consultas de bajo stock y actualizaciones de unidades disponibles.

El proyecto cumple con la carga de datos desde CSV, la incorporación de registros adicionales, la implementación de los 15 servicios requeridos, la generación de resultados y la validación reproducible del sistema.

10. Bibliografía

Corbellini, A., Mateos, C., Zunino, A., Godoy, D. y Schiaffino, S. (2017). *Persisting big-data: The NoSQL landscape*.

Perkins, L., Redmond, E. y Wilson, J. R. (2018). *Seven Databases in Seven Weeks: A Guide to Modern Databases and the NoSQL Movement*. Segunda edición.

Date, C. J. (2003). *An Introduction to Database Systems*. Octava edición.